CHAPTER 8

Instruction Pipelining & Hazards

Stages · speedup · stall counting · structural, data, control hazards · forwarding · branch penalty

QUICK TRICK

k stages, n instructions: **cycles = k + n − 1**. Speedup = **n·k / (k + n − 1)** → k. Clock = **slowest stage**. Pipelined CPI = **1 + stall cycles per instruction**. RAW without forwarding: **2 stalls**; with forwarding: **0** (ALU→ALU), **1** (load-use).

8.1 The idea

● **LEVEL 1 · BEGINNER**

The problem. Executing one instruction fully before starting the next leaves most hardware idle. **The idea.** Overlap the instructions, like an assembly line: while one instruction is being executed, the next is being decoded and a third fetched.

Analogy. A laundry with washer, dryer and folder. Start the next load as soon as the washer is free, not after the whole first load is folded.

Classic 5-stage pipeline: **IF** (fetch) → **ID** (decode, read registers) → **EX** (ALU) → **MEM** (data memory) → **WB** (write back).

8.2 Time and speedup

● **LEVEL 2 · EXAM**

```
Cycle for pipelined clock   = max(stage delay) (+ latch delay)
Time for n instructions     = (k + n - 1) x clock
Non-pipelined time          = n x (sum of stage delays)
Speedup (equal stages)      = n*k / (k + n - 1)   ->  k as n grows
Efficiency = speedup / k      Throughput = n / ((k + n - 1) x clock)
```

Example. 5 stages with delays 10, 8, 12, 9, 6 ns. Non-pipelined: 45 ns per instruction. Pipelined clock = 12 ns. For 100 instructions: pipelined (5 + 99) × 12 = **1248 ns**; non-pipelined 100 × 45 = 4500 ns; speedup ≈ **3.6** (not 5, because the stages are unbalanced).

QUICK TRICK

Speedup can never exceed the number of stages k. Unbalanced stages and stalls both reduce it. With ideal CPI = 1: **speedup = k / (1 + stalls per instruction)** (same clock).

8.3 Hazards

● **LEVEL 2 · EXAM**

Hazard	Cause	Cure
Structural	Two instructions need the same hardware in one cycle (e.g. one memory for IF and MEM)	Duplicate resources (split I/D cache), or stall
Data	An instruction needs a result not yet written. **RAW** (true), WAR, WAW	Forwarding (bypass), stalls, compiler scheduling
Control	Branch target not known until later	Stall, predict, delayed branch

In the simple in-order 5-stage pipeline only **RAW** hazards happen. WAR and WAW appear with out-of-order execution.

Counting RAW stalls (5-stage, register file written in the first half-cycle and read in the second)

```
I1: ADD R1,R2,R3       I2: SUB R4,R1,R5   (I2 needs R1)
No forwarding: I1 writes R1 in WB (cycle 5); I2 can read it in cycle 5.
cycle:  1  2  3  4  5  6  7  8
I1      IF ID EX ME WB
I2         IF ID ID ID EX ME WB       -> 2 stall cycles
With forwarding (EX result of I1 -> EX of I2): 0 stalls.
```

Producer → consumer (adjacent)	No forwarding	With forwarding
ALU → ALU	2 stalls	**0**
Load → use (load-use)	2 stalls	**1** (data exists only after MEM)
Two apart	1 stall	0

Three-instruction example. I1: LW R1,0(R2); I2: ADD R3,R1,R4; I3: SUB R5,R3,R6. **With forwarding:** I2 stalls 1 (load-use); I3 needs no stall. Total = (5 + 3 − 1) + 1 = **8 cycles**. **Without forwarding:** I2 stalls 2, I3 stalls 2 → 7 + 4 = **11 cycles**.

8.4 Control hazards

● LEVEL 2 · EXAM

- Branch penalty = cycles lost until the branch outcome and target are known (resolved in EX: 2 cycles; in ID: 1 cycle).
- **Policies:** stall always · predict not-taken (penalty only when taken) · predict taken · **delayed branch** (fill the slot after the branch with a useful instruction) · dynamic prediction (1-bit, 2-bit counters).
- **CPI = 1 + (branch frequency) × (fraction of branches paying the penalty) × (penalty)**. Example: 20% branches, 60% taken, predict not-taken, penalty 2: CPI = 1 + 0.2 × 0.6 × 2 = **1.24**.
- **Structural example:** single memory port, 30% loads/stores each stall instruction fetch by 1 cycle: CPI = 1 + 0.3 = **1.3**.

8.5 Common mistakes

COMMON MISTAKE

- Using n × k cycles instead of k + n − 1 for a pipeline.
- Dividing the non-pipelined time by the **sum** of stages for the clock (the clock is the **max** stage).
- Forgetting the 1-cycle load-use stall even with forwarding.
- Counting WAR/WAW hazards in a simple in-order pipeline.
- Forgetting latch/register delay when the question gives it (add it to every stage).

QUICK TRICK

Memory line: "First instruction pays k cycles; every later one adds just one."

GATE QUICK CHECK — Chapter 8

```
cycles = k+n-1      speedup = nk/(k+n-1) <= k      clock = max stage (+latch)
CPI = 1 + stalls/instr      speedup = k/(1+stalls)
RAW adjacent: no fwd 2 stalls | fwd ALU 0 | fwd load-use 1
branch CPI = 1 + f_branch * f_penalised * penalty
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Cycles to execute 10 instructions in a 5-stage pipeline (no stalls).

Q2. Maximum possible speedup of a k-stage pipeline.

Q3. Which hazards can occur in a simple in-order 5-stage pipeline?

Q4. RAW stalls between adjacent ALU instructions with forwarding.

● LEVEL 2 · EXAM

Q5. Speedup for n = 100 instructions on a 5-stage pipeline with equal stages. Efficiency?

Q6. Stage delays 10, 8, 12, 9, 6 ns. Time for 100 instructions pipelined and speedup over non-pipelined?

Q7. Adjacent RAW dependence, no forwarding, register file split-cycle. Stall cycles?

Q8. 20% branches, 60% of them taken, predict not-taken with penalty 2. CPI?

● LEVEL 3 · GATE

Q9. LW R1,0(R2); ADD R3,R1,R4; SUB R5,R3,R6 with forwarding. Total cycles?

Q10. Same code without forwarding (split-cycle register file). Total cycles?

Q11. A single-ported memory causes 1 stall cycle for every load/store, which are 30% of instructions. CPI?

Q12. Stages need 200, 150, 300, 250, 100 ps and each pipeline register adds 20 ps. Clock cycle and speedup for a long program (vs non-pipelined 1000 ps)?

Answers

Q	Solution
1	5 + 10 − 1 = **14**.
2	**k**.
3	Structural, RAW data hazards and control hazards (WAR/WAW only with out-of-order execution).
4	**0**.
5	Speedup = 500/104 ≈ **4.81**; efficiency = 4.81/5 ≈ **96.2%**.
6	Clock 12 ns; (5 + 99) × 12 = **1248 ns**. Non-pipelined 100 × 45 = 4500 ns. Speedup ≈ **3.6**.
7	**2**.
8	1 + 0.2 × 0.6 × 2 = **1.24**.
9	I2 stalls 1 (load-use), I3 none: (5 + 3 − 1) + 1 = **8**.
10	I2: 2 stalls, I3: 2 stalls: 7 + 4 = **11**.
11	1 + 0.3 × 1 = **1.3**.
12	Clock = 300 + 20 = **320 ps**. Speedup for large n = 1000/320 ≈ **3.1** (non-pipelined has no register overhead).

Next: Chapter 9 — Memory Hierarchy, Performance and Interfacing.